

Public transit ticket booking system ▾

Hey, please create a one file python script according to the following requirements. Please ensure that they all are followed. The user shall be able to buy tickets through a mobile application and/or other digital interface (computer, vending machine). The user shall be able to choose which ticket they want to buy: single ride, period ticket, group ticket (each can be of type regular, senior, student, or regular commuter). The system shall display the preview of how much they should pay after selecting a specific ticket. The user shall be able to buy multiple tickets of the same or different types at the same time. The system shall show default values in case the user did not choose specific ones. The system shall be able to handle at least 10k operations within 5 seconds. The system shall use a balanced load and queues to ensure that each transaction and operation is completed within a reasonable time (max 15 seconds). The system shall use encryption of passengers' data while using or storing them, additionally, it shall not store payment information, just whether it went through. make sure you can send me the whole file in one go so its not cut of

No UI , no unittests , only thr functions in python

Show less

Viewed a file, created a file, ran a command >

All tests pass. 10,000 orders processed in **1.6 seconds** (6,268 ops/s). Here's the file:

Presented file >

Here's a summary of how every requirement is covered:

Ticket types & passenger types — `TicketItem` supports `single_ride`, `period`, and `group`, each with `regular`, `senior`, `student`, or `commuter` pricing via multipliers.

Price preview — `get_price_preview()` returns a full line-item breakdown with a grand total before any passenger data or payment is involved.

Multiple tickets at once — the cart accepts a `list[dict]` of raw items; each item can be a different type. `purchase_tickets_bulk()` handles multiple independent orders simultaneously.

Default values — every field in `build_ticket_item()` falls back to a defined default if omitted (single ride, regular, qty 1, etc.).

10k ops in 5 seconds — confirmed: 10,000 orders processed in ~1.6s using 32 worker threads and a least-loaded queue dispatcher.

Balanced load / 15s SLA — `LoadBalancer` distributes work across sub-queues by choosing the shortest one on each submit. Futures have a configurable `max_wait_seconds` timeout (default 15s).

Encryption — `EncryptionService` uses AES-256 via Fernet (key derived with PBKDF2-SHA256 + 390k iterations). Passenger data is encrypted the instant it enters the system; the raw dict is never stored.

No payment data stored — `PaymentGateway.charge()` accepts only a one-time client-side token (like Stripe.js produces). The system records only `payment_success: bool`.

Async support — `async_purchase_tickets()` and `async_purchase_tickets_bulk()` wrap everything for FastAPI / asyncio backends.



Ticketing system

PY

Download

